



Gestión del proyecto

18 de Julio 2026

Tabla de contenido

1. Introducción.....	4
2. Metodología de trabajo	4
2.1 Organización del equipo.....	4
2.2 Integrantes	4
2.3 Roles y responsabilidades	4
2.4 Distribución por microservicios.....	5
2.5 Herramientas utilizadas.....	5
2.6 Ceremonias	5
3. Definition of Ready y Definition of Done	6
3.1 Definition of Ready	6
3.2 Definition of Done.....	6
4. Gestión del backlog	6
4.1 Estrategia de estimación.....	6
4.2 Product backlog	7
4.3 Priorización.....	7
5. Planeación y ejecución de sprints.....	7
5.1 Planeación general.....	7
5.2 Sprints y objetivo de cada uno.....	7
5.3 Sprint backlog.....	7
6. Evidencias del proceso.....	8
6.1 Evidencias de pull requests.....	8
6.2 Evidencias de releases.....	8
7. Métricas del proyecto	8
7.1 Velocidad	8
7.2 Burndown	8
7.3 Historias completadas y trasladadas.....	8
7.4 Bugs	9
7.5 Cycle time y lead time.....	9

8. Riesgos y cambios de alcance.....	9
▶ 8.1 Riesgos del proyecto	9
8.2 Acciones de mitigación	9
8.3 Cambios de alcance	9
9. Lecciones aprendidas	10
10. Estado final del proyecto	10
10.1 Estado final	10
10.2 Enlace a Jira	10

1. Introducción

Este documento describe cómo el equipo de ECIExpress planeó, ejecutó y controló el proyecto a lo largo de sus seis sprints, desde la definición del backlog hasta el cierre. ECIExpress es una plataforma de marketplace universitario para la Escuela Colombiana de Ingeniería Julio Garavito, construida como un sistema de microservicios con billetera digital recargable, gestión de vendedores, notificaciones en tiempo real y logística de entrega.

2. Metodología de trabajo

El equipo trabajó bajo un esquema ágil adaptado a la disponibilidad real de sus integrantes, con reuniones semanales en lugar de dailies diarios, y seguimiento del trabajo en Jira, proyecto SCRUM, tipo team-managed, organizado en épicas por microservicio.

2.1 Organización del equipo

El equipo estuvo conformado por 5 personas. La responsabilidad de mantenimiento y corrección general de todos los microservicios recayó principalmente en el desarrollador backend, con apoyo del ingeniero de IA. Los dos desarrolladores full stack se dividieron el trabajo de frontend junto con un microservicio cada uno, uno se encargó de Identity y Fulfillment junto con frontend, y el otro se encargó de Order junto con frontend.

2.2 Integrantes

Integrante	Rol
Esteban Aguilera Contreras	Ingeniero de IA
Elizabeth Correa Suárez	Desarrolladora full stack
Sebastián Ortega	Desarrollador full stack
Sebastián Velásquez	Desarrollador backend
Marlio Charry	Desarrollador frontend

2.3 Roles y responsabilidades

Desarrollador backend, Sebastián Velásquez: mantenimiento y corrección general de todos los microservicios del proyecto.

Ingeniero de IA, Esteban Aguilera Contreras: desarrollo de la IA de la plataforma, los modelos 3D y el módulo de productos, con apoyo al desarrollador backend en el mantenimiento general.

Desarrolladora full stack, Elizabeth Correa Suárez, y **desarrollador full stack**, Sebastián Ortega: desarrollo de frontend, y de los microservicios Identity, Fulfillment y Order repartidos entre ambos.

Desarrollador frontend, Marlio Charry: desarrollo de la interfaz de usuario de la plataforma.

2.4 Distribución por microservicios

El backlog en Jira se organizó en 7 épicas, una por microservicio: Identity and Admin, Product, Order, Financial, Notification, Fulfillment y Reporting. El bus de eventos asíncrono y el API Gateway no tienen épica propia, su trabajo se repartió como tareas dentro de los microservicios que los consumen.

2.5 Herramientas utilizadas

- Jira, para backlog, historias, tareas y seguimiento de sprints, proyecto SCRUM.
- GitHub, como repositorio de código de cada microservicio y del frontend.
- Discord, como canal de comunicación del equipo.
- Claude Code, para tareas de implementación delegadas mediante documentos de instrucciones, y para auditoría de seguridad y rendimiento del código.

2.6 Ceremonias

El equipo trabajó con reuniones semanales (weeklies) en lugar de ceremonias diarias, dada la disponibilidad del equipo.

Lunes: planning, definición del trabajo de la semana.

Viernes: retroalimentación del equipo, revisión de avances y de cómo estuvo la semana.

Sábado: reunión con el tutor del proyecto, para correcciones y validación de avances.

3. Definition of Ready y Definition of Done

Las definiciones propuestas por el equipo son las siguientes, definition of ready y definition of done.

3.1 Definition of Ready

- La historia tiene una descripción clara del comportamiento esperado.
- La historia está asociada a la épica del microservicio correspondiente.
- Se identificaron los eventos o endpoints involucrados.
- Se identificaron las dependencias con otros microservicios, si existen.
- El equipo entiende el criterio de aceptación antes de iniciar el sprint.

3.2 Definition of Done

- Código implementado y subido al repositorio del microservicio correspondiente.
- Migraciones de base de datos creadas cuando aplica, con la opción synchronize siempre en false.
- Flujo probado manualmente o mediante pruebas automatizadas.
- Documentación OpenAPI actualizada cuando aplica.
- Eventos publicados siguiendo la convención servicio punto entidad punto acción, con el envelope estándar del proyecto.
- Revisado en la retroalimentación de los viernes o validado por el tutor los sábados.
- Desplegado o probado en el entorno de desarrollo sin errores.

4. Gestión del backlog

4.1 Estrategia de estimación

La estimación del trabajo se hizo por horas, sobre tareas atómicas. Cada subtarea promediaba entre 2 y 3 horas de trabajo. Con las 47 subtareas del backlog, esto da un total base de entre 94 y 141 horas, más algunas horas adicionales en las tareas más pesadas, como el despliegue a producción o la configuración de infraestructura, que superaban ese promedio.

4.2 Product backlog

El backlog se estructuró en jerarquía Epic, Historia y Subtarea. Cada una de las 7 épicas de microservicio agrupa las historias de usuario necesarias para cubrir su funcionamiento, y cada historia tiene sus subtareas técnicas asociadas. En total el backlog llegó a 7 épicas, 32 historias y 47 subtareas.

4.3 Priorización

La priorización siguió las dependencias técnicas entre microservicios. Identity se priorizó primero porque los demás servicios dependen de autenticación y gestión de usuarios. Luego Product y la base de Order, después Financial, luego Notification y Fulfillment, y al cierre Reporting junto con las tareas transversales de pruebas, documentación y despliegue.

5. Planeación y ejecución de sprints

5.1 Planeación general

El proyecto se planeó en 6 sprints de una semana cada uno, del 4 de junio al 16 de julio de 2026.

5.2 Sprints y objetivo de cada uno

Sprint	Fechas	Objetivo
Sprint 1	4 al 11 de junio	Identity and Admin completo, más infraestructura base del proyecto.
Sprint 2	11 al 18 de junio	Product completo, más base de Order: modelo de pedido, máquina de estados y chat.
Sprint 3	18 al 25 de junio	Financial completo, más QR de retiro parcial en Fulfillment.
Sprint 4	25 de junio al 2 de julio	Notification completo, más comunicación en tiempo real.
Sprint 5	2 al 9 de julio	Fulfillment completo, Reporting y API Gateway.
Sprint 6	9 al 16 de julio	Pruebas de integración, pruebas de carga, corrección de bugs, documentación y despliegue.

5.3 Sprint backlog

Cada sprint se compuso de las historias y subtareas de las épicas correspondientes según la tabla anterior. El detalle historia por historia se puede consultar directamente en el tablero de Jira del proyecto, clave SCRUM.

6. Evidencias del proceso

Como evidencia del proceso de desarrollo, el proyecto cuenta con los repositorios de GitHub de cada microservicio y del frontend, con el historial de commits y pull requests correspondiente a cada sprint. No se llevó un registro fotográfico separado de las reuniones de planning, dailies, revisiones o retrospectivas.

6.1 Evidencias de pull requests

Disponibles en el historial de cada repositorio de GitHub del proyecto, uno por microservicio y uno para el frontend.

6.2 Evidencias de releases

Disponibles en el historial de despliegues de cada repositorio y en el registro de commits de la rama principal de cada microservicio.

7. Métricas del proyecto

Tipo de issue	Total	Finalizado	Por hacer
Épicas	7	7	0
Historias	32	32	0
Subtareas	47	45	2

7.1 Velocidad

Con un promedio de 2.5 horas por subtaska sobre las 47 subtareas del backlog, el total estimado es de aproximadamente 118 horas de trabajo. Repartidas en los 6 sprints del proyecto, esto da una velocidad estimada de aproximadamente 19.6 horas por sprint, equivalente a unas 7.8 subtareas por sprint.

7.2 Burndown

Con la velocidad estimada anterior, un burndown ideal del proyecto consumiría de forma aproximadamente lineal unas 19 a 20 horas de trabajo por semana a lo largo de las 6 semanas, hasta llegar a cero al cierre del Sprint 6.

7.3 Historias completadas y trasladadas

Las 32 historias del backlog están marcadas como Finalizado en Jira. No hay historias trasladadas a un sprint adicional.

7.4 Bugs

El proyecto no usó un tipo de issue independiente para bugs en Jira, las correcciones se manejaron como subtareas dentro de la historia de corrección de bugs y ajustes finales del Sprint 6.

7.5 Cycle time y lead time

El cycle time estimado, el tiempo efectivo de trabajo por tarea, es de 2 a 3 horas, el mismo promedio de duración de una subtask atómica. El lead time estimado, desde que la tarea entraba al sprint el lunes de planning hasta que se cerraba, es de aproximadamente 3 a 4 días, considerando que la mayoría de tareas se completaban antes de la retroalimentación de los viernes.

8. Riesgos y cambios de alcance

8.1 Riesgos del proyecto

- Concurrencia en el manejo de eventos asíncronos entre microservicios.
- Disponibilidad continua del módulo de IA de la plataforma.
- Cambios de alcance sobre piezas visuales del proyecto, en particular el modelo 3D de los edificios del campus.

8.2 Acciones de mitigación

- Concurrencia por eventos: se adoptó un envelope estándar para todos los eventos del bus, con idempotencyKey y correlationId, para evitar procesamiento duplicado y poder rastrear un evento a través de los microservicios.
- Disponibilidad de la IA y escalabilidad de los microservicios: quedan identificados como riesgos abiertos, sin una acción de mitigación aplicada todavía, ver sección de riesgos residuales.

8.3 Cambios de alcance

Se removió el modelado 3D individual de cada edificio del campus, porque requería modelado prácticamente manual de cada estructura. Se mantuvo el mapa 3D general

del campus con extrusiones por polígono a partir del GeoJSON del campus, sin el detalle de fachada por edificio en esta etapa.

9. Lecciones aprendidas

- No crear microservicios nuevos para asuntos transversales, logging, monitoreo, caché, throttling o notificaciones, esas responsabilidades pertenecen a infraestructura o a un servicio ya existente.
- Los tokens SAS deben ser de corta duración, de un solo blob y una sola operación, generados bajo demanda por el microservicio dueño del recurso, nunca por el API Gateway.
- En TypeORM, synchronize debe quedar siempre en false, todo cambio de esquema pasa por migraciones.
- El webhook de Wompi debe aceptar el cuerpo crudo como Record de tipo desconocido, un ValidationPipe con whitelist en true puede eliminar campos antes de verificar la firma y romper la validación.
- La consistencia del payload de eventos entre microservicios necesita estandarizarse entre equipos, el servicio de Identity envuelve sus datos en un campo payload, lo que obligó a aplicar el patrón defensivo data.payload si no, data en los consumidores.
- Dejar un solo método de validación de identidad para todos los microservicios, en lugar de que cada uno implemente el suyo, salvo el de Wompi, que se deja como está por ser la validación de firma del proveedor de pagos. Esto va de la mano con respetar el principio de responsabilidad única, y no poner a un microservicio a resolver algo que no le corresponde.

10. Estado final del proyecto

10.1 Estado final

El proyecto está cerrado en su totalidad a nivel de historias, con las 7 épicas y las 32 historias del backlog marcadas como Finalizado en Jira. Quedan 2 subtarefas abiertas, la ejecución de pruebas automatizadas y un pentesting asistido por herramientas, no manual, como actividades de cierre del proyecto.

10.2 Enlace a Jira

El proyecto se gestionó en [Jira](#) bajo la clave de proyecto SCRUM, nombre EciExpress.